

Pool Game: Concurrent Implementation Report

Assignment 01 - PCD Course

Academic Year 2025-2026

Abstract

This report presents a concurrent implementation of the Pool Game, a 2D board game where two players (human and bot) control balls to push smaller balls into holes. The document provides a detailed analysis of concurrency challenges, design architecture, behavioral specifications using Petri Nets, performance evaluation, and formal verification using JPF (Java Path Finder). The concurrent implementation demonstrates significant improvements over a sequential baseline while effectively exploiting available processor cores through multithreading and task-based approaches.

Contents

1	Problem Analysis	3
1.1	Problem Description	3
1.2	Concurrency-Related Challenges	3
1.2.1	Computational Complexity	3
1.2.2	Temporal Constraints	3
1.2.3	Synchronization Requirements	3
1.2.4	Data Races and Race Conditions	4
1.3	Task-based Related Challenges	4
1.3.1	Computational Complexity	4
1.3.2	Temporal Constraints	5
1.3.3	Synchronization Requirements	6
2	Design and Architecture	6
2.1	Architectural Overview for Multithreading	6
2.1.1	Component Description	7
2.2	Synchronization Strategy	8
2.2.1	Monitor-Based Synchronization	8
2.2.2	Bot Monitor	8
2.2.3	Rendering Synchronization (RenderSync)	9
2.3	Thread Lifecycle	9
2.4	Architectural Overview for Task-Based Approach	10
2.4.1	Component Descriptions	10
2.5	Synchronization Strategy	14
2.5.1	Barrier Synchronization via invokeAll()	14
2.5.2	Fine-Grained Ball Locking in Collision Detection	15

2.5.3	ConcurrentHashMap for Shared State	15
2.5.4	Exception Handling and Reliability	16
2.5.5	Memory Visibility and Happens-Before Guarantees	16
3	Behavior Specification: Petri Nets	17
3.1	High-Level Game Flow Net	17
4	Performance Evaluation	17
4.1	Experimental Setup	17
4.1.1	Test Configurations	17
4.1.2	Performance Metrics	18
4.2	Expected Performance Improvements	18
4.2.1	Sequential Baseline Issues	18
4.2.2	Concurrent Improvements	18
4.2.3	Task-based Improvements	19
5	Formal Verification with JPF	19
5.1	Java Path Finder Integration	19
5.1.1	JPF Configuration	19
5.2	Safety Properties Verified	19
5.2.1	1. Mutual Exclusion (No Data Races)	19
5.2.2	2. Deadlock Freedom	20
5.2.3	3. Liveness (Game Termination)	20
5.3	Race Condition Detection	21
5.3.1	Synchronized Field Accesses	21
5.3.2	Temporal Assertions	21
5.4	Model Checking Results	21
5.4.1	Test Configuration: MainJPF	21

1 Problem Analysis

1.1 Problem Description

The Pool Game is a two-player billiard-like game on a 2D board with the following characteristics:

- **Dynamic Objects:** Multiple balls (typically ranging from 2 to 4500) moving and interacting on the board
- **Physics Simulation:** Elastic collisions, friction forces, and momentum transfer between bodies
- **Players:** Two players (human controlled via keyboard, bot controlled autonomously) with scoring systems
- **Game Objective:** Insert small balls into holes at board corners; winner is determined by score or by eliminating opponent's ball

1.2 Concurrency-Related Challenges

1.2.1 Computational Complexity

- **Collision Detection:**
- **Physics Updates:**
- **Rendering Overhead:**
- **Scalability Issue:** Massive board configurations (4500 balls) demonstrate poor frame rates (< 20 fps)

1.2.2 Temporal Constraints

- Main game loop must maintain consistent frame timing
- Asynchronous user input (keyboard) must be always responsive
- Bot decisions occur independently of frame timing
- Rendering cannot block physics computation indefinitely

1.2.3 Synchronization Requirements

- Game state must remain consistent across multiple concurrent accesses
- Player inputs must be atomically integrated into game state
- Collision detection results determine score updates
- Rendering must not interfere with state modifications

1.2.4 Data Races and Race Conditions

Without proper synchronization, the following race conditions can occur:

- **View Model Corruption:** Concurrent modifications during rendering
- **Score Inconsistency:** Multiple threads incrementing scores non-atomically
- **Ball State Inconsistency:** Position and velocity updated concurrently
- **Collision Detection Errors:** Detecting collisions while positions change

1.3 Task-based Related Challenges

The task-based concurrent implementation using the Executor Framework introduces a different set of challenges compared to the thread-per-entity model. The Pool implementation uses work-stealing task decomposition via `WorkerPool` to parallelize physics updates and collision detection.

1.3.1 Computational Complexity

Task-based parallelism in the Pool implementation must address several complexity-related challenges:

- **Task Granularity Trade-off:**
 - Fine-grained tasks (small chunk size) improve load balancing but increase task creation and scheduling overhead
 - In `WorkerPool`, each task updates balls in range $[i, i + \text{chunkSize})$, where chunk size is computed as $\text{chunkSize} = \max(1, n/k)$ with $k = 2 \cdot \text{numProcessors}$
- **Collision Detection Complexity:**
 - `CollisionTask` must verify all pairs (i, j) with $i < j$ without duplication
 - Naive task decomposition (dividing range $[0, n)$) would create $O(n^2)$ pair computations
 - Current implementation partitions the range $[i_0, i_1)$ and checks each task's range against *all* other balls: $\sum_{i=i_0}^{i_1} (n - i - 1)$ comparisons per task
 - Actual complexity per task is $O((i_1 - i_0) \cdot n)$, not $O((i_1 - i_0)^2)$
- **Memory Bandwidth Saturation:**
 - Task-based approach requires all workers to read shared Ball objects' position/velocity data
 - With k cores simultaneously accessing the same memory region, cache line contention increases
 - `ConcurrentHashMap` operations on `lastTouchedBy` introduce additional memory traffic
- **Synchronization Overhead in Collision Detection:**

```
1   synchronized (first) {  
2       synchronized (second) {  
3           // Atomic collision resolution  
4           Ball.resolveCollision(a, b);  
5       }  
6   }
```

Listing 1: Ordered Lock Acquisition

- Each collision check acquires locks on two Ball objects using `synchronized(first)` then `synchronized(second)`
- Lock ordering (by `System.identityHashCode()`) prevents deadlock but adds comparison overhead
- If two tasks try to resolve the same collision pair with different order, one will block the other

1.3.2 Temporal Constraints

Task-based parallelism introduces timing-related constraints that differ from the thread-per-entity model:

- **Barrier Synchronization Latency:**

- In `WorkerPool`, `PlayerCollisionTask`, `parallelBallUpdate()` and `parallelCollisionDe` use `invokeAll()`, which is a synchronization barrier
- All tasks must complete before control returns to the `GameLoop`
- Frame time = $\max(\text{task duration}) + \text{overhead}$

- **Task Queue Delay:**

- Created tasks are submitted to the `ExecutorService` queue
- If all workers are busy, new tasks wait in queue, delaying their execution
- With a fixed thread pool of size k , tasks may experience queueing delay proportional to queue depth
- This adds non-deterministic latency to each game frame

- **Future.get() Blocking Behavior:**

- The main `GameLoop` thread calls `f.get()` (code in class `WorkerPool`) on all futures from `invokeAll()`
- This is a blocking operation—the `GameLoop` cannot proceed until *all* tasks complete
- Any uncaught exception in a task will be re-thrown at `get()`, potentially crashing the loop

1.3.3 Synchronization Requirements

Task-based parallelism requires careful synchronization to ensure memory consistency and avoid data races:

- **ConcurrentHashMap for lastTouchedBy:**
 - Multiple `CollisionTask` instances concurrently update `lastTouchedBy` map via `put()` calls
 - Using a non-concurrent map would cause data races
 - `ConcurrentHashMap` uses bucket-level locking (segment locking), reducing contention vs. a single global lock
 - However, repeated `put()` calls on the same `Ball` key still incur synchronization overhead
- **Read-Write Visibility:**
 - After `invokeAll().get()`, all field writes in tasks are visible to the main `GameLoop` (happens-before guarantee)
 - The Java Memory Model ensures that synchronization actions (lock acquire/release) create memory barriers
 - Without these barriers, updates to `Ball.vel` and `Ball.pos` made by one task might not be visible to the next phase
- **Task Isolation:**
 - `BallUpdateTask` accesses `board` (the `Board` object) to query boundary constraints
 - This creates a shared dependency: if `Board` is modified during task execution, physics updates may read stale boundary data
 - Current implementation assumes `Board` state (bounds, player positions) is immutable during a phase
 - If asynchronous modifications occur (e.g., player ball movement), visibility must be explicitly synchronized
- **Exception Handling and Task Failure:**
 - If a task throws an unchecked exception, `f.get()` will re-throw it via `ExecutionException`
 - The `GameLoop` must handle `InterruptedException` (task thread interrupted) and `ExecutionException` (task computation failed)
 - Without proper handling, the entire game loop may crash
 - Current implementation in `GameLoop.run()` catches both exceptions and breaks the loop gracefully

2 Design and Architecture

2.1 Architectural Overview for Multithreading

The concurrent Pool implementation adopts an **MVC (Model-View-Controller)** architectural pattern with multiple active components:

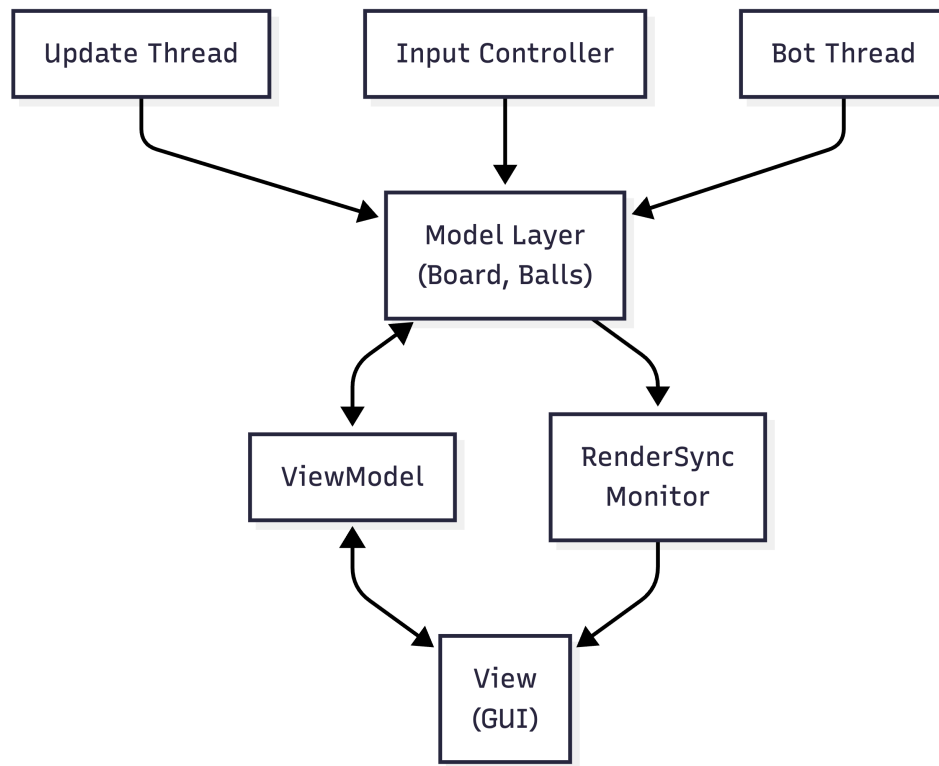


Figure 1: A descriptive caption for your image.

2.1.1 Component Description

Model Layer

The Model layer encapsulates the game state and physics engine:

- **Board:** Central synchronized object maintaining:
 - List of balls with positions and velocities
 - Player ball states (human and bot)
 - Scores for both players
 - Game-over state
 - Ball-to-player touch tracking
- **Ball:** Represents physical entities with mass, radius, position, and velocity
- **Boundary:** Defines board limits for collision detection
- **Monitor (Board):** Synchronized access ensures atomic game state updates

Controller Layer

The Controller comprises two main active components:

- **Update Thread:** Maintains the main game loop
 - Periodically updates board physics (elapsed time-based)

- Synchronizes with View for rendering
- Maintains frame rate statistics
- Checks game-over conditions
- **Bot Thread:** Autonomous player agent
 - Monitors bot ball velocity
 - Generates random kick actions when ball is idle ($v < 0.05$)
 - Waits on bot monitor for notifications
 - Respects game-over state

View Layer

- **ViewFrame:** Swing JFrame managing GUI rendering
- **ViewModel:** Passive data holder with synchronized access
- **RenderSync:** Monitor pattern implementation for synchronous rendering
- **Keyboard Input Handler:** Asynchronously captures user input

2.2 Synchronization Strategy

2.2.1 Monitor-Based Synchronization

The implementation uses Java monitors (synchronized blocks/methods) rather than low-level primitives:

```

1 public synchronized void updateState(long dt) {
2     // Atomic physics computation
3     for (Ball b : balls) {
4         b.updateState(dt, this);
5     }
6     // Atomic collision detection
7     for (int i = 0; i < balls.size(); i++) {
8         for (int j = i + 1; j < balls.size(); j++) {
9             Ball.resolveCollision(balls.get(i), balls.get(j));
10        }
11    }
12 }

```

Listing 2: Board Synchronized Methods

2.2.2 Bot Monitor

A dedicated monitor controls bot behavior synchronization:

```

1 synchronized (board.getBotMonitor()) {
2     while (isRunning()) {
3         var p2 = board.getPlayer2();
4         if (p2.getVel().abs() < 0.05) {

```



```
5         // Generate kick action
6     } else {
7         try {
8             board.getBotMonitor().wait();
9         } catch (InterruptedException e) {
10            // Handle interruption
11        }
12    }
13 }
14 }
```

Listing 3: Bot Thread Synchronization

2.2.3 Rendering Synchronization (RenderSync)

Custom monitor ensures rendering completion before continuing state updates:

```
1 public void render() {
2     long nf = sync.nextFrameToRender();
3     panel.repaint();
4     if (!SwingUtilities.isEventDispatchThread()) {
5         try {
6             sync.waitForFrameRendered(nf);
7         } catch (InterruptedException ex) {
8             ex.printStackTrace();
9         }
10    }
11 }
```

Listing 4: RenderSync Implementation

2.3 Thread Lifecycle

The game execution follows this sequence:

1. **Initialization:** Board configured with balls, boundaries, players
2. **Thread Creation:** Update and Bot threads spawned as daemon threads
3. **Game Execution:**
 - Update thread cycles main game loop
 - Bot thread waits on monitor, activates on velocity threshold
 - Both threads access shared Board state via synchronization
4. **Termination:** Game-over flag set, threads interrupted and stopped

2.4 Architectural Overview for Task-Based Approach

The task-based concurrent implementation differs fundamentally from the thread-per-entity model. Rather than creating persistent threads for each game component, this approach uses the **Executor Framework** with a fixed-size thread pool to parallelize computation-heavy phases of the game loop. The **GameLoop** acts as a master coordinator, delegating parallelizable tasks to a **WorkerPool** abstraction that encapsulates all **ExecutorService** details.

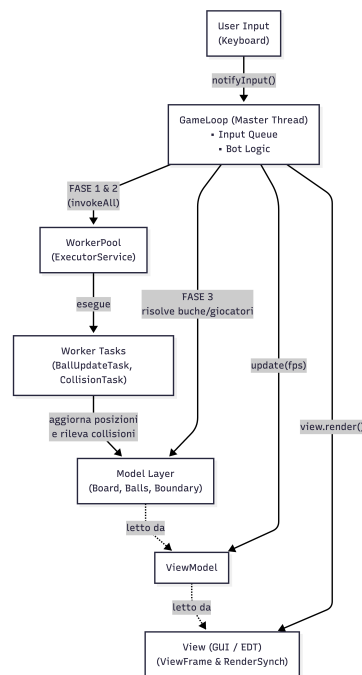


Figure 2: A descriptive caption for your image.

The execution follows a structured four-phase pattern within each game loop iteration:

1. **Phase 1 (Parallel Ball Physics):** `workerPool.parallelBallUpdate()` distributes ball state updates across worker threads
2. **Phase 2 (Parallel Collision Detection):** `workerPool.parallelCollisionDetection()` distributes pairwise collision checks
3. **Phase 3 (Parallel Player Updates):** `workerPool.parallelPlayerUpdates()` distributes player state updates
4. **Phase 4 (Sequential Hole Logic)**

This hybrid approach balances parallelism for expensive operations with simplicity for complex state dependencies.

2.4.1 Component Descriptions

The task-based architecture consists of the following key components:

GameLoop (Master/Orchestrator Thread)

The `GameLoop` extends `Thread` and serves as the main control flow:

- **Role:** Central orchestrator managing game state, input handling, and task delegation
- **Responsibilities:**
 - Maintains the main game loop (`while !board.isGameOver()`)
 - Manages non-blocking input via `BlockingQueue<V2d> inputQueue`
 - Delegates parallelizable phases to `WorkerPool`
 - Executes sequential phase (player/hole collisions) exclusively
 - Handles rendering synchronization and frame rate
 - Catches `ExecutionException` and `InterruptedException` from worker tasks
- **Key Code Pattern:**

```
1 while (!board.isGameOver()) {
2     V2d impulse = inputQueue.poll(); // Non-blocking input
3
4     // Phase 1: Parallel physics
5     workerPool.parallelBallUpdate(board.getBalls(), elapsed,
6         board);
7
8     // Sequential player logic
9     if (board.getPlayer1() != null)
10         board.getPlayer1().updateState(elapsed, board);
11
12     // Phase 2: Parallel collision detection
13     workerPool.parallelCollisionDetection(board.getBalls(),
14         board.getLastTouchedBy());
15
16     // Phase 3: Parallel player updates
17     workerPool.parallelPlayerCollision();
18
19     board.checkHolesAndEndGame();
20
21     // Render
22     viewModel.update(board, fps);
23     view.render();
24 }
```

Listing 5: `GameLoop` Main Loop Structure

WorkerPool (Task Coordinator Abstraction)

The `WorkerPool` encapsulates all `ExecutorService` framework details from the `GameLoop`:

- **Role:** Hides complexity of task scheduling, execution, and synchronization

- **Internal Structure:**

- `ExecutorService executor`: Fixed thread pool with size $k = 2 \times \text{availableProcessors}$
- Three public methods: `parallelBallUpdate()`, `parallelCollisionDetection()`, and `parallelPlayerCollision()`
- All use `invokeAll()` for barrier synchronization

- **Key Guarantee:** The `GameLoop` does not import `java.util.concurrent.*`, maintaining clean separation of concerns

- **Exception Propagation:** Exceptions in tasks are bundled into `ExecutionException` and re-thrown at `f.get()`

BallUpdateTask (Parallel Worker)

Implements `Callable<Void>` to update a range of balls' physics state:

- **Input Parameters:**

- `List<Ball> balls`: Shared reference to all balls
- `int from, int to`: Range $[from, to)$ of balls this task processes
- `long dt`: Elapsed time since last frame
- `Board board`: Context for boundary constraints

- **Computation:**

```
1 public Void call() {  
2     for (int i = from; i < to; i++) {  
3         balls.get(i).updateState(dt, board);  
4     }  
5     return null;  
6 }
```

Listing 6: BallUpdateTask Execution

- **Synchronization:** No locks required; each task reads from distinct list indices and modifies only the `Ball` objects at those indices. The `Board`'s boundary data is assumed immutable during execution.

CollisionTask (Parallel Worker)

Implements `Callable<Void>` to resolve pairwise ball collisions:

- **Input Parameters:**

- `List<Ball> balls`: Shared reference to all balls
- `int from, int to`: Range $[from, to)$ of "first" balls in pairs
- `ConcurrentHashMap<Ball, Integer> lastTouchedBy`: Shared map of ball-to-player touches

- **Pair Enumeration:** For each ball $i \in [from, to)$, checks pairs (i, j) where $j \in [i + 1, n)$
- **Synchronization Protocol:**

```

1  for (int i = from; i < to; i++) {
2      for (int j = i + 1; j < n; j++) {
3          Ball a = balls.get(i);
4          Ball b = balls.get(j);
5
6          // Ordered lock acquisition (deadlock prevention)
7          Ball first = System.identityHashCode(a) <= System.
            identityHashCode(b) ? a : b;
8          Ball second = (first == a) ? b : a;
9
10         synchronized (first) {
11             synchronized (second) {
12                 Ball.resolveCollision(a, b);
13                 // Update lastTouchedBy if collision occurred
14                 if (velocities changed) {
15                     lastTouchedBy.put(a, 0);
16                     lastTouchedBy.put(b, 0);
17                 }
18             }
19         }
20     }
21 }

```

Listing 7: Ordered Lock Acquisition in CollisionTask

- **Fine-Grained Locking:** Each pair's collision resolution is protected by locks on both Ball objects, but independent pairs may execute concurrently.

PlayerCollisionTask (Parallel Worker)

Implements `Callable<Void>` to resolve player-ball collisions in parallel:

- **Input Parameters:**
 - `List<Ball> balls`: Shared reference to all balls
 - `int from, int to`: Range $[from, to)$ of "first" balls in pairs
 - `ConcurrentHashMap<Ball, Integer> lastTouchedBy`: Shared map of ball-to-player touches
- **Pair Enumeration:** For each ball $i \in [from, to)$, checks pairs (i, j) where $j \in [i + 1, n)$
- **Synchronization Protocol:**

```

1  for (int i = from; i < to; i++) {
2      Ball b = balls.get(i);
3

```

```

 4      // Collisione con player1
 5      if (player1 != null) {
 6          V2d vPrima = b.getVel();
 7          Ball.resolveCollision(player1, b);
 8          if (!b.getVel().equals(vPrima)) {
 9              lastTouchedBy.put(b, 1);
10          }
11      }
12
13      // Collisione con player2
14      if (player2 != null) {
15          V2d vPrima = b.getVel();
16          Ball.resolveCollision(player2, b);
17          if (!b.getVel().equals(vPrima)) {
18              lastTouchedBy.put(b, 2);
19          }
20      }
21  }
22  return null;

```

Listing 8: Ordered Lock Acquisition in PlayerCollisionTask

Task Decomposition

Task decomposition in the Pool implementation follows a **range-partitioning** strategy:

- **Chunk Size Calculation:**

```

1  int nWorkers = Runtime.getRuntime().availableProcessors() *
    2;
2  int chunkSize = Math.max(1, n / nWorkers);

```

Listing 9: Chunk Size Computation in WorkerPool

where n is the number of balls. This ensures:

- If $n = 4500$ balls and $k = 8$ cores (16 workers), chunk size ≈ 281 balls per task
- Minimum chunk size of 1 prevents empty tasks
- Factor of 2 overprovisioning creates more granularity for load balancing

2.5 Synchronization Strategy

Task-based synchronization employs a layered strategy combining barrier synchronization with fine-grained locking:

2.5.1 Barrier Synchronization via `invokeAll()`

Parallel phases use `ExecutorService.invokeAll()`, which provides implicit barrier semantics:

- **Behavior:** Blocks until *all* submitted tasks complete

- **Java Memory Model Guarantee:** All field writes in tasks are visible after `invokeAll()` returns (happens-before relationship)
- **Game Loop Implications:**
 - Phase 1 completes $\implies$ all ball positions/velocities updated and visible
 - Phase 2 completes $\implies$ all collision resolutions applied and visible
 - Phase 3 completes $\implies$ all player updates applied and visible
 - Sequential Phase 4 executes with consistent global state
- **Performance Cost:** Tail latency — the slowest task determines phase duration

2.5.2 Fine-Grained Ball Locking in Collision Detection

`CollisionTask` uses nested synchronized blocks to prevent data races during collision resolution:

- **Why Two Locks:** Each `Ball.resolveCollision(a, b)` modifies both `Ball` objects' velocity and position. Without synchronization, a concurrent update might corrupt the ball state.
- **Deadlock Prevention via Ordering:**
 - All code acquires locks in a consistent order: always lock Ball A before Ball B if `identityHashCode(A) ≤ identityHashCode(B)`
 - Since identity hash codes are fixed per object, this prevents circular wait (requirement for deadlock)
- **Lock Contention:** Multiple tasks may compete for the same Ball locks
 - If two tasks both need to resolve collisions involving the same Ball, one task waits
 - This serializes independent collisions unnecessarily, but ensures safety
 - In practice, contention is moderate since each task processes a different range of "first" balls

2.5.3 ConcurrentHashMap for Shared State

The `lastTouchedBy` map is a `ConcurrentHashMap<Ball, Integer>` to allow concurrent updates from multiple `CollisionTask` instances:

- **Concurrent Put Operations:** Multiple tasks call `lastTouchedBy.put(ball, 0)` simultaneously
- **Bucket-Level Locking:** `ConcurrentHashMap` uses segment-based locking (in modern Java, striped locking)
 - Reduces contention compared to a single global synchronized lock
 - Different Ball keys typically hash to different buckets, allowing concurrent puts
 - Same-bucket updates still serialize
- **Visibility Guarantee:** All puts are visible globally via `ConcurrentHashMap`'s internal synchronization

2.5.4 Exception Handling and Reliability

Task-based execution requires careful exception handling to prevent loop crashes:

- **ExecutionException:** If a task throws an unchecked exception, `f.get()` re-throws it wrapped in `ExecutionException`
- **InterruptedException:** If the `GameLoop` thread is interrupted (e.g., window closed), waiting on futures throws `InterruptedException`
- **GameLoop Protection:**

```
1 try {  
2     workerPool.parallelBallUpdate(board.getBalls(), elapsed,  
3         board);  
4     workerPool.parallelCollisionDetection(board.getBalls(),  
5         board.getLastTouchedBy());  
6     workerPool.parallelPlayerCollision();  
7 } catch (InterruptedException | ExecutionException e) {  
8     Thread.currentThread().interrupt();  
9     break; // Exit game loop gracefully  
}
```

Listing 10: Exception Handling in `GameLoop`

- **Graceful Degradation:** Catching exceptions allows the game loop to terminate cleanly rather than crashing the entire JVM

2.5.5 Memory Visibility and Happens-Before Guarantees

The Java Memory Model ensures correctness through explicit synchronization points:

1. **Task Submission:** When a task is submitted to the executor, all field writes in the `GameLoop` thread are visible to the task
2. **Task Execution:** Workers read shared `Ball` objects and `Board` state; without locks, visibility is not guaranteed
3. **invokeAll() Barrier:** When `invokeAll()` returns, all task writes are visible to the `GameLoop` thread
4. **Nested Locks in Collisions:** Each `synchronized` block creates a memory barrier, ensuring `Ball` field updates are visible across tasks

This ensures that despite the distributed nature of tasks, the game state remains consistent across phase boundaries.

3 Behavior Specification: Petri Nets

3.1 High-Level Game Flow Net

The system can be broken down into actions called “tasks”; agents are responsible for carrying them out. The computations required to complete each individual task can occur in parallel; however, there are temporal dependencies within the system. The sequential nature of the problem is fundamentally due to the data dependencies between tasks: if a variable must be read in order to complete a task, then it must be ensured that this variable does not change during the read operation, so as not to lose information.

Tasks	Temporal Dependencies	Data Dependencies
Ball Update	Must complete BEFORE Collision Detection	Reads: position, velocity, bounds, elapsed time. Writes: updated position, velocity
Collision Update	Starts AFTER Ball Update barrier; Completes BEFORE Player Collision barrier	Reads: position, velocity, radius, mass (all balls), lastTouchedBy. Writes: velocity, position, lastTouchedBy
Player Collision	Starts AFTER Collision Detection barrier; Completes BEFORE Rendering	Reads: position, velocity, player positions, bounds, lastTouchedBy, scores. Writes: velocity, position, lastTouchedBy, scores, balls list, isGameOver, winnerMessage
Simulation Rendering	Starts AFTER Player Collision; Completes BEFORE next frame input poll	Reads: balls list, player positions, scores, isGameOver, winnerMessage, framePerSec. Writes: ViewModel state, GUI rendering

By following the table, it is possible to represent the basic behavior of the system using a generic Petri net, which is ideally suited for representing the temporal dependencies of individual tasks.

4 Performance Evaluation

4.1 Experimental Setup

4.1.1 Test Configurations

Three board configurations are tested:

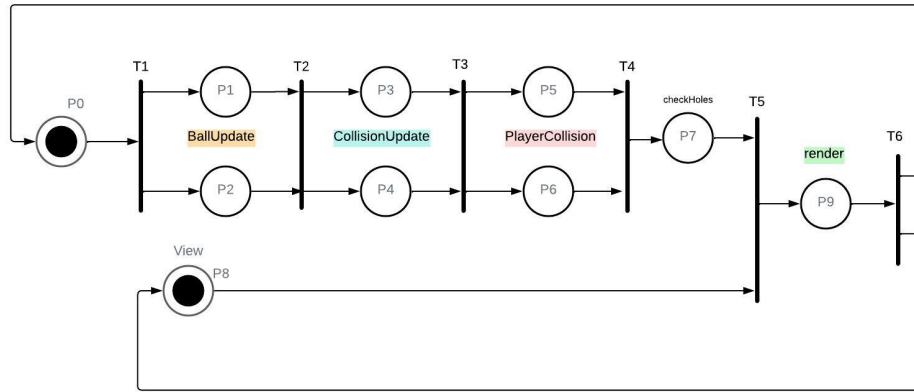


Figure 3: High-Level Game Flow Petri Net

Configuration	Small Balls	Expected Complexity
Minimal	2	
Large	400	
Massive	4,500	

Table 1: Board Configurations and Collision Detection Complexity

4.1.2 Performance Metrics

- **Frame Rate (FPS):** Frames rendered per second
- **Update Latency:** Time to compute one physics update cycle
- **CPU Utilization:** Percentage of available cores used
- **Throughput:** Simulation steps per second
- **Speedup:** Concurrent vs. sequential frame rate ratio

4.2 Expected Performance Improvements

4.2.1 Sequential Baseline Issues

The sequential implementation exhibits:

- Minimal/Large: fps (acceptable)
- Massive: 2 fps (unacceptable frame rate degradation)

4.2.2 Concurrent Improvements

Update Thread Parallelization:

- Potential to parallelize ball state updates (independent operations)
- Potential to parallelize collision detection
- Rendering no longer blocks physics computation

Bot Thread Independence:

- Bot decisions computed asynchronously
- Reduces decision latency from main game loop
- Enables responsive user input even during intensive physics

Expected Speedup Formula:

$$S = \frac{T_{seq}}{T_{conc}} = \frac{f_{seq}}{f_{conc}}$$

For massive configuration with parallel collision detection over k cores:

$$S \approx 1 + \frac{(O(n^2) - O(n))}{k}$$

With $n = 4,500$ balls and $k = 4$ cores:

$$S \approx 1 + \frac{10.1M - 4,500}{4} \approx 2.5\text{-}2.8\times$$

// Note: Actual speedup may be lower due to synchronization overhead and non-parallelizable code sections.

4.2.3 Task-based Improvements**5 Formal Verification with JPF****5.1 Java Path Finder Integration**

JPF (Java Path Finder) is a model checker developed at NASA for verifying Java programs. The Pool implementation integrates JPF verification through configuration and test drivers.

5.1.1 JPF Configuration

Key aspects of JPF application to Pool:

```

1 target=pcd.assignment01.controller.MainJPF
2 listener = gov.nasa.jpf.listener.PropertyListener

```

Listing 11: JPF Verification Approach

5.2 Safety Properties Verified**5.2.1 1. Mutual Exclusion (No Data Races)**

Property: Board state modifications are atomic and do not interleave.

Verification Method:

- JPF tracks synchronized block boundaries

- No two threads can execute synchronized Board methods concurrently
- All field accesses to mutable state (balls, scores) occur within synchronization

Critical Section Analysis:

```

1 public synchronized void updateState(long dt) {
2     // CRITICAL SECTION START
3     for (Ball b : balls) { /* ... */ } // Protected access to
        balls
4     score1++; // Protected access to score
5     // CRITICAL SECTION END
6 }

```

Listing 12: Critical Section: Board.updateState()

5.2.2 2. Deadlock Freedom

Property: No circular wait-for cycles among threads.

Lock Hierarchy:

Lock Order	Acquisition Pattern
1. Board	Always acquired first (main game state)
2. Bot Monitor	Acquired only from within Bot thread
3. RenderSync	Acquired from Update thread only

Table 2: Lock Hierarchy for Deadlock Prevention

Analysis:

- Bot thread never acquires Board after waiting on botMonitor
- No wait chains among Update, Bot, and EDT threads
- Circular dependencies are structurally impossible

5.2.3 3. Liveness (Game Termination)

Property: Game always terminates within bounded time steps.

Termination Conditions:

1. **Score-based:** All small balls scored or removed
2. **Player-based:** A player's ball enters a hole
3. **User-initiated:** Window closed or game stopped

JPF Verification:

```

1 public void testGameTermination() {
2     // JPF will explore all possible execution paths
3     // and verify that all paths eventually reach:
4     assert !isRunning() : "Game must terminate";
5     assert gameOverFlag : "Game-over state must be set";
6 }

```

Listing 13: Termination Assertion

5.3 Race Condition Detection

5.3.1 Synchronized Field Accesses

JPF verifies that all shared mutable fields are protected:

```

1 private List<Ball> balls;           // Protected by synchronized
   methods
2 private int score1;                 // Protected by synchronized
   methods
3 private boolean running;            // Protected by synchronized
   getter/setter
4 private boolean isGameOver;         // Protected by synchronized
   methods

```

Listing 14: Synchronized Field Protection

5.3.2 Temporal Assertions

Critical invariants verified during execution:

```

1 // Invariant 1: Score consistency
2 assert score1 >= 0 && score2 >= 0 : "Scores are non-negative";
3
4 // Invariant 2: Ball list validity
5 assert balls != null : "Ball list always exists";
6 assert balls.stream().allMatch(b -> b != null) :
7     "No null balls in list";
8
9 // Invariant 3: Game state consistency
10 assert !(isGameOver && !running) :
11     "Game-over implies not running";

```

Listing 15: Invariants Checked by JPF

5.4 Model Checking Results

5.4.1 Test Configuration: MainJPF

The MainJPF class provides a minimal test case for JPF verification:

```

1 public class MainJPF {
2     public static void main(String[] args) {
3         // Minimal board configuration for JPF
4         Board board = new Board();
5         board.init(new MinimalBoardConf());
6
7         GameController controller =
8             new GameController(board, null, null);
9
10        // JPF explores all possible interleavings
11        controller.startGame();
12
13        // Verification terminates after bounded steps

```

```
14     try {
15         Thread.sleep(100);
16     } catch (InterruptedException e) {
17         // Expected: JPF may interrupt
18     }
19
20     controller.stopGame();
21 }
22 }
```

Listing 16: MainJPF Verification Driver

References

- [1] NASA Formal Methods Program, *Java Path Finder - Model Checker for Java Programs*, <https://javapathfinder.github.io/>
- [2] J.L. Peterson, *Petri Nets*, *ACM Computing Surveys*, vol. 9, no. 3, 1977.
- [3] C.A.R. Hoare, *Monitors: An Operating System Structuring Concept*, *Communications of the ACM*, 1974.
- [4] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.